

Car Rental Management System

System Analysis and Design Document

Content

1. General Information
2. Executive Summary
3. System Request
4. Workplan
5. Feasibility Analysis
6. Requirements Definition
 - 6.1. Functional Requirements
 - 6.2. Non-Functional Requirements
7. Process Modeling
 - 7.1. Data Flow Diagram (DFD)
8. Data Modeling
 - 8.1. Entity Relationship Diagram (ERD)
9. Functional Model
 - 9.1. Use Case Diagram
 - 9.2. Use Case Descriptions
 - 9.3. Activity Diagram
10. Structural Models
 - 10.1. CRC Cards
 - 10.2. Class Diagram
11. Transition to Design
 - 11.1. Package Diagram
 - 11.2. Package Diagram overview
12. Model Verification & Validation
 - 12.1. Model Consistency Checklist
13. Appendices

1. General Information

Project Title: Car Rental Management System

Project Code: CRMS-2026

Document Version: 1.0

Prepared By: Mohammed Sami, Taha Al-Muslimi, Ahmed Al-Maari

Date: January 13, 2026

Organization: Code & Cognition

Project Type: Database Management System for Car Rental Operations

Technology Platform: SQL Database with Web-based Interface

Project Domain: Transportation and Vehicle Rental Services

2. Executive Summary

The Car Rental Management System is designed to streamline and automate the entire vehicle rental process from customer registration through vehicle return. The system addresses critical operational challenges in managing customer information, vehicle inventory, booking processes, payment transactions, and vehicle returns in an integrated digital platform.

businesses struggle with manual processes that lead to inefficiencies in tracking vehicle availability, managing customer bookings, calculating rental fees, and processing returns. The proposed system will provide a comprehensive solution that maintains real-time vehicle inventory, automates rental calculations, tracks mileage consumption, and ensures accurate financial transactions.

The system encompasses five core functional areas: customer management with complete personal and licensing information, vehicle information management including specifications and rental rates, vehicle booking with detailed reservation tracking, rental transaction processing with payment logging, and vehicle return management with automatic calculation of fees and refunds. The database schema includes entities for customers, vehicles, vehicle categories, fuel types, rental bookings, rental transactions, vehicle returns, and maintenance records.

Expected benefits include improved operational efficiency, reduced errors in rental calculations, enhanced customer service through faster processing, better vehicle utilization tracking, and comprehensive financial reporting capabilities. The system will support multiple fuel types (gasoline, diesel, electric, hybrid) and various vehicle categories (4x4, sedan, etc.) to accommodate diverse rental fleets.

3. System Request

Element	Description	Examples for Car Rental System
Project Sponsor	The person who initiates the project and who serves as the primary point of contact on the business side	Car Rental Company CEO Operations Manager IT Department Head Finance Director
Business Need	The business-related reason for initiating the system	Improve rental process efficiency Reduce manual errors in booking and billing Enhance vehicle tracking and utilization Improve customer service delivery Streamline payment processing and refunds Enable real-time vehicle availability management
Business Requirements	The business capabilities that the system will provide	Capture and maintain customer personal information and driver's license data Maintain comprehensive vehicle inventory with specifications, mileage, and rental rates Process vehicle bookings with pickup/dropoff locations and rental periods Log rental transactions with payment details Calculate actual rental costs with additional charges Track vehicle mileage consumption Generate rental reports and financial summaries Support multiple fuel types and vehicle categories Automate refund calculations for early returns
Business Value	The benefits that the system will create for the organization	25% reduction in booking processing time 15% increase in vehicle utilization rates \$50,000 annual savings from reduced administrative overhead 99% accuracy in rental fee calculations Elimination of 3 FTE manual data entry positions 30% improvement in customer satisfaction scores Real-time visibility of fleet availability Reduction in billing disputes by 40%

Special Issues or Constraints	Issues that are relevant to the implementation of the project and decisions made by the committee about the project	System must comply with local data protection regulations for customer information Driver's license verification integration required Must support Arabic and English languages Integration with existing payment gateways needed Mobile-responsive interface required for field operations Minimum 99.5% system uptime during business hours Data migration from legacy systems must be completed within 2 weeks Staff training must be completed before go-live date
--------------------------------------	---	---

4. Workplan

4.1 Project Overview

The workplan compresses all documentation deliverables into a 7-day intensive schedule, focusing exclusively on analysis and design documentation rather than development. Parallel work streams and dedicated team focus enable rapid completion of all required documents

4.2 Work Breakdown Structure (WBS)

Day	Documentation Deliverables	Duration	Key Activities
Day 1	1. General Information 2. Executive Summary 3. System Request	1 day	Project definition, sponsor identification, executive summary creation, document business needs, requirements, and value proposition
Day 2	4. Workplan 5. Feasibility Analysis	1 day	Develop final workplan, conduct technical/economic/operational/schedule feasibility studies
Day 3	6. Requirements Definition 6.1 Functional Requirements 6.2 Non-Functional Requirements	1 day	Extract and document functional requirements (Customer, Vehicle, Booking, Transaction, Return management) and non-functional requirements (security, performance, usability)
Day 4	7. Process Modeling 7.1 Data Flow Diagram (DFD)	1 day	Create context diagram, Level 0 and Level 1 DFDs for customer registration, vehicle booking, payment, and return processes
Day 5	8. Data Modeling 8.1 Entity Relationship Diagram (ERD)	1 day	Develop ERD including Customer, Vehicle, VehicleCategories, FuelTypes, RentalBooking, RentalTransaction, VehicleReturns, and Maintenance entities with relationships and keys
Day 6	9. Functional Model 9.1 Use Case Diagram 9.2 Use Case Descriptions 9.3 Activity Diagram 10. Structural Models 10.1 CRC Cards 10.2 Class Diagram	1 day	Design use case diagrams for actors (Rental Agent, Customer, System Admin), describe major use cases (Create Booking, Record Payment, Return Vehicle), create activity diagrams, develop CRC cards and class diagram aligned with ERD
Day 7	11. Model Verification & Validation 11.1 Model Consistency Checklist 12. Transition to Design 12.1 Package Diagram 13. Appendices	1 day	Verify consistency across DFD, ERD, and use cases; complete validation checklist; create package diagram for system layers (Presentation, Business, Data Access); compile appendices (data dictionary, glossary)

5. Feasibility Analysis

5.1 Technical Feasibility

Criteria	Details
Technology Availability	SQL database technology is mature and widely available for implementing the relational schema
Technical Expertise	Standard web development and database skills are readily available; team can complete analysis and design documentation within 7 days
Hardware Requirements	Standard server infrastructure sufficient; no specialized hardware needed
Software Compatibility	System can integrate with existing payment gateways and third-party services
Scalability	Database design supports growth in customers, vehicles, and transactions
Development Timeline	Documentation can be completed in 7 days with focused team effort

5.2 Economic Feasibility

Cost-Benefit Analysis (7-Day Documentation Phase)

Category	Amount	Notes
Project Manager (16 hours \$75/hr)	\$1,200	Days 1-2
Business Analyst (56 hours \$60/hr)	\$3,360	Days 1-7
Financial Analyst (8 hours \$65/hr)	\$520	Day 2
Database Designer (16 hours \$70/hr)	\$1,120	Days 5-6
System Analyst (32 hours \$70/hr)	\$2,240	Days 4-7
QA Lead (8 hours \$60/hr)	\$480	Day 7
System Architect (8 hours \$80/hr)	\$640	Day 7
Technical Writer (8 hours \$50/hr)	\$400	Day 7
Total Documentation Costs	\$9,960	7 days

Full Project Cost Estimate

Category	Year 1	Year 2	Year 3	Total (3 Years)
Documentation (Complete)	\$9,960	\$0	\$0	\$9,960
Development Costs	\$65,000	\$0	\$0	\$65,000
Infrastructure Costs	\$15,000	\$8,000	\$8,000	\$31,000
Training Costs	\$5,000	\$2,000	\$2,000	\$9,000
Maintenance Costs	\$10,000	\$12,000	\$12,000	\$34,000
Total Costs	\$104,960	\$22,000	\$22,000	\$148,960

Expected Benefits

Benefit Category	Year 1	Year 2	Year 3	Total (3 Years)
Labor Cost Savings (automated booking/returns)	\$30,000	\$35,000	\$40,000	\$105,000
Reduced Billing Errors (accurate calculations)	\$15,000	\$18,000	\$20,000	\$53,000
Improved Vehicle Utilization	\$25,000	\$30,000	\$35,000	\$90,000
Reduced Customer Complaints	\$10,000	\$12,000	\$15,000	\$37,000
Total Benefits	\$80,000	\$95,000	\$110,000	\$285,000

6. Requirements Definition

The Car Rental System must support end-to-end management of customers, vehicles, bookings, rental transactions, and vehicle returns, including mileage tracking and additional charges calculation. The system will provide a centralized database and user interface for rental staff to perform all rental operations efficiently.

6-1 Functional Requirements

6.1. Customer Management

6.1.1 The system shall store customer personal information including full name, contact information, and driver's license number.

6.1.2 The system shall allow staff to create new customer records with all required personal details.

6.1.3 The system shall allow staff to search, view, edit, and deactivate existing customer records.

6.1.4 The system shall prevent creating duplicate active customer records with the same driver's license number.

6.1.5 The system shall validate driver's license number format during customer registration.

6.2. Vehicle Information Management

6.2.1 The system shall maintain comprehensive vehicle information including make, model, year, mileage, fuel type, plate number, vehicle category, and rental price per day.

6.2.2 The system shall support multiple fuel types: gasoline (petrol), diesel, electric, and hybrid.

6.2.3 The system shall support multiple vehicle categories such as 4x4, sedan, and other types.

6.2.4 The system shall allow staff to add new vehicles, update vehicle details, and modify availability status.

6.2.5 The system shall record and maintain current mileage for each vehicle.

6.2.6 The system shall track vehicle availability status (IsAvailableForRent) to prevent double-booking.

6.2.7 The system shall allow recording maintenance operations for vehicles including description, maintenance date, and cost.

6.3. Vehicle Booking

6.3.1 The system shall allow staff to create rental bookings linking a customer to a specific vehicle.

6.3.2 The system shall record booking information: customer ID, vehicle ID, rental start date, rental end date, pickup location, and drop-off location.

6.3.3 The system shall record initial rental days and initial total due amount at the time of booking.

6.3.4 The system shall calculate the initial total due amount based on rental days and rental price per day.

6.3.5 The system shall record initial vehicle check notes documenting the vehicle condition at pickup.

6.3.6 The system shall prevent double-booking of vehicles for overlapping rental periods.

6.3.7 The system shall allow staff to view and search bookings by customer, vehicle, date range, and booking status.

6.3.8 The system shall allow staff to cancel or modify bookings according to business rules.

6.4. Rental Transaction (Payment Processing)

6.4.1 The system shall create a rental transaction record associated with each booking when the customer makes initial payment.

6.4.2 The system shall log payment details including payment method and transaction information.

6.4.3 The system shall record the initial paid amount (PaidInitialTotalDueAmount) at the time of payment.

6.4.4 The system shall store transaction date and updated transaction date for audit tracking.

6.4.5 The system shall maintain fields for actual total due amount, total remaining, and total refunded amount.

6.4.6 The system shall allow staff to update transactions when bookings are modified or additional charges apply.

6.4.7 The system shall provide a summary of amounts due, paid, remaining, and refunded for each transaction.

6.5. Vehicle Return Processing

6.5.1 The system shall allow staff to process vehicle returns linked to existing bookings.

- 6.5.2** The system shall record the actual return date and calculate actual rental days.
- 6.5.3** The system shall record final vehicle check notes documenting the vehicle condition at return.
- 6.5.4** The system shall specify and record any additional charges (e.g., damage, late fees).
- 6.5.5** The system shall record the vehicle's current mileage at return and calculate consumed mileage during the rental period.
- 6.5.6** The system shall calculate the actual final amount due based on actual rental days and additional charges.
- 6.5.7** The system shall update the original rental transaction with actual total due amount and calculate remaining or refunded amounts.
- 6.5.8** The system shall process refunds when customers return vehicles early or when overpayment occurs.
- 6.5.9** The system shall update the vehicle's availability status and current mileage after return processing.

6-2 Non-Functional Requirements

6.2 Performance

- 6.2.1** The system shall display search results for customers, vehicles, and bookings within 3 seconds under normal load.
- 6.2.2** The system shall support concurrent use by up to 20 rental staff users without noticeable performance degradation.
- 6.2.3** The system shall process booking creation and transaction updates within 2 seconds.

6.3 Reliability and Availability

- 6.3.1** The system shall maintain 99.5% uptime during business operating hours.

6.3.2 The system shall ensure zero data loss for financial transactions and booking records through proper transaction management.

6.3.3 The system shall provide automated daily database backups with retention for at least 30 days.

6.3.4 The system shall support recovery from failures within 2 hours.

6.4 Security

6.4.1 The system shall require authenticated login credentials for all users.

6.4.2 The system shall enforce role-based access control to protect sensitive operations and data.

6.4.3 The system shall protect customer personal information and driver's license data in compliance with data protection regulations.

6.4.4 The system shall encrypt sensitive data (passwords, payment details) both in transit and at rest.

6.4.5 The system shall log all critical actions including booking creation, transaction updates, refunds, and data deletions.

6.4.6 The system shall automatically lock user accounts after 5 failed login attempts.

6.5 Usability

6.5.1 The system shall provide a user-friendly interface that can be learned by new rental staff within 2 days of training.

6.5.2 The system shall use clear labels, consistent navigation, and intuitive forms for all rental processes.

6.5.3 The system shall validate mandatory fields and display clear error messages for incomplete or invalid data.

6.5.4 The system shall support both English and Arabic language interfaces.

6.5.5 The system shall provide context-sensitive help and tooltips for complex operations.

6.6 Maintainability and Extensibility

6.6.1 The system architecture shall allow adding new vehicle categories and fuel types without major code changes.

6.6.2 The system shall support integration with external payment gateways and third-party services.

6.6.3 The system's database schema shall be fully documented with ERD and data dictionary.

6.6.4 The system shall follow modular design principles to facilitate future enhancements.

6.7 Portability and Deployment

6.7.1 The system shall be deployable on standard Windows/Linux server infrastructure.

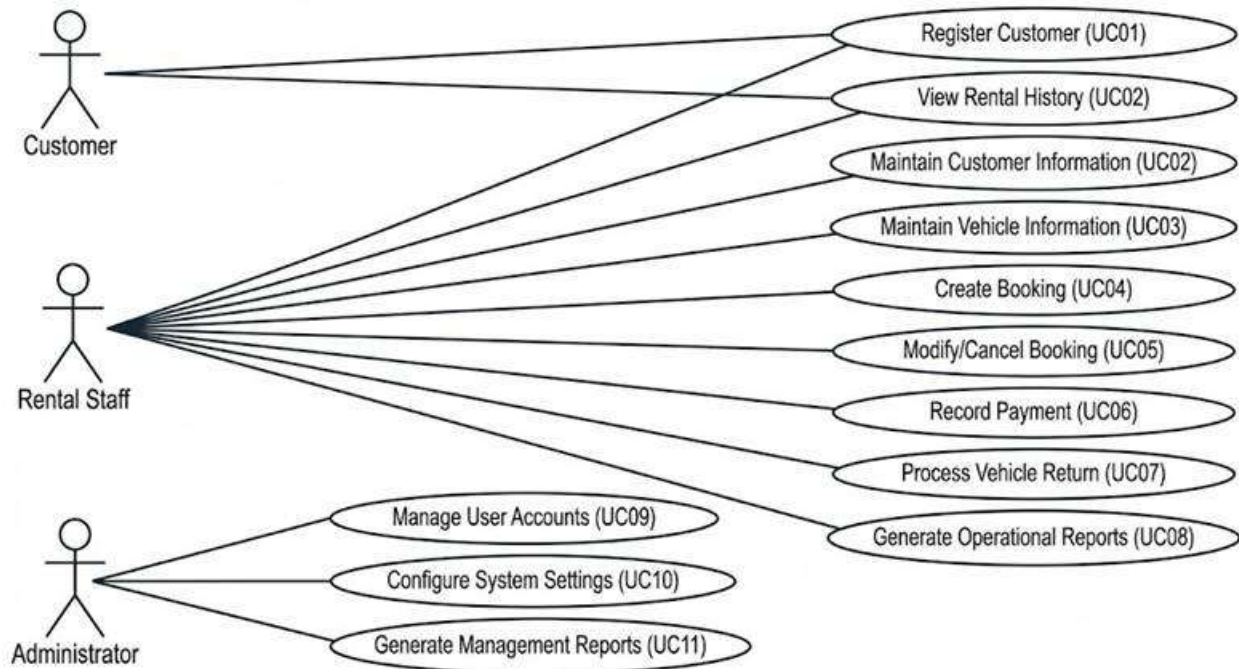
6.7.2 The system shall support browser-based access compatible with modern web browsers (Chrome, Firefox, Edge, Safari).

6.7.3 The system shall be accessible from multiple workstations within the rental office network.

6.7.4 The system shall support responsive design for access from desktop and tablet devices.

9. Functional Model

9.1. Use Case Diagram



9.2. Use Case Descriptions

UC01 – Register Customer

Attribute	Description
Use Case Name	Register Customer
Actor	Rental Staff
Precondition	Staff is logged into the system
Main Flow	1. Staff selects "Register Customer" 2. System displays the customer registration form 3. Staff enters customer full name, contact information, and driver's license number 4. System validates that all mandatory fields are filled and driver's license number format is correct 5. System checks that no active customer exists with the same driver's license number 6. System saves the new customer record 7. System confirms that the customer has been registered successfully
Post Condition	A new customer record is stored in the database with personal information and driver's license number
Alternative Flow	If required information is missing, system displays an error message and asks staff to complete the fields
Exception Handling	If duplicate driver's license is detected, system displays an error and prevents saving the record

UC02 – Maintain Customer Information

Attribute	Description
Use Case Name	Maintain Customer Information
Actor	Rental Staff
Precondition	Customer exists in the system; Staff is logged into the system
Main Flow	1. Staff selects "Search Customer" 2. Staff enters search criteria (name, phone, or driver's license) 3. System displays a list of matching customers 4. Staff selects a customer from the list 5. System displays customer details 6. Staff edits the required fields or chooses to deactivate the customer 7. System validates the updated information 8. System saves the changes and confirms successful update
Post Condition	Customer data is updated or customer is deactivated
Alternative Flow	If no matching customers found, system displays "No results" message
Exception Handling	If validation fails, system displays specific error message and prevents saving

UC03 – Maintain Vehicle Information

Attribute	Description
Use Case Name	Maintain Vehicle Information
Actor	Rental Staff
Precondition	Staff is logged into the system
Main Flow	1. Staff selects "Vehicle Management" 2. Staff chooses to add a new vehicle or search for an existing vehicle 3. For a new vehicle, staff enters make, model, year, mileage, fuel type, category, plate number, and rental price per day 4. System validates mandatory fields and plate number uniqueness 5. System saves the vehicle record and sets its availability status 6. For existing vehicles, staff updates details (e.g., mileage, price, status) and saves changes
Post Condition	Vehicle information is stored and vehicle availability status is up to date
Alternative Flow	If plate number already exists, system suggests editing the existing vehicle instead
Exception Handling	If mandatory fields are missing, system prevents saving and displays error message

UC04 – Create Booking

Attribute	Description
Use Case Name	Create Booking
Actor	Rental Staff
Precondition	Customer exists in the system; Vehicle is available for the requested period; Staff is logged in
Main Flow	1. Staff selects "Create Booking" 2. Staff searches and selects an existing customer 3. Staff searches and selects an available vehicle 4. Staff enters rental start date, rental end date, pickup location, and drop-off location 5. System calculates initial rental days and initial total due amount based on rental price per day 6. Staff enters initial vehicle check notes (condition at pickup) 7. System checks for overlapping bookings for the same vehicle 8. System saves the booking and confirms successful creation

Post Condition	A booking record is stored with initial rental days, initial total due amount, and vehicle check notes
Alternative Flow	If no vehicles available, system notifies staff and suggests changing dates or vehicle category
Exception Handling	If overlapping booking found, system displays a conflict message and prevents saving

UC05 – Modify/Cancel Booking

Attribute	Description
Use Case Name	Modify/Cancel Booking
Actor	Rental Staff
Precondition	Booking exists in the system; Staff is logged in
Main Flow	1. Staff selects "Manage Bookings" 2. Staff searches and selects a booking 3. System shows booking details 4. Staff chooses to modify dates, vehicle, locations, or to cancel the booking 5. System re-validates vehicle availability and recalculates initial total due amount if dates/vehicle change 6. System saves the updated booking or marks it as cancelled
Post Condition	Booking details are updated or booking is cancelled and status is changed
Alternative Flow	If modification creates a conflict, system suggests alternative vehicles or dates
Exception Handling	If booking already has payment or return recorded, system restricts certain modifications

UC06 – Record Payment (Create Rental Transaction)

Attribute	Description
Use Case Name	Record Payment (Create Rental Transaction)
Actor	Rental Staff
Precondition	Booking exists and is active; Staff is logged in
Main Flow	1. Staff selects "Record Payment" for a chosen booking 2. System displays booking details and initial total due amount 3. Staff enters payment method and amount paid 4. System records payment details and sets the initial paid amount 5. System creates rental transaction with transaction date 6. System updates total remaining amount field 7. System confirms successful payment recording
Post Condition	A rental transaction is stored with payment details, initial paid amount, and transaction date
Alternative Flow	If partial payment is made, system records the amount and calculates remaining balance
Exception Handling	If payment amount exceeds due amount, system displays warning and asks for confirmation

UC07 – Process Vehicle Return

Attribute	Description
Use Case Name	Process Vehicle Return
Actor	Rental Staff
Precondition	An active booking and transaction exist for the vehicle; Staff is logged in
Main Flow	1. Staff selects "Process Return" 2. Staff searches and selects the active booking/vehicle 3. Staff enters actual return date and current vehicle mileage 4. System calculates actual rental days and consumed mileage 5. Staff records final vehicle check notes and any additional charges (e.g., damage, late fees) 6. System calculates actual final amount due including additional charges 7. System updates rental transaction with actual total due amount, remaining amount, or refund amount 8. System updates vehicle availability status and current mileage 9. System confirms successful return processing
Post Condition	Return information is stored, additional charges and consumed mileage are calculated, transaction is updated, and vehicle is available again
Alternative Flow	If early return, system calculates refund; if late return, system calculates additional charges
Exception Handling	If additional charges result in outstanding balance, system flags for follow-up payment

UC08 – Generate Operational Reports

Attribute	Description
Use Case Name	Generate Operational Reports
Actor	Rental Staff
Precondition	Staff is logged in
Main Flow	1. Staff selects "Reports" 2. Staff chooses a report type (e.g., bookings for a date range, returns, payments) 3. Staff enters filters (date range, vehicle, customer, status) 4. System retrieves and displays the report 5. Staff may export the report to PDF or Excel if needed
Post Condition	Selected report is generated and displayed (and optionally exported)
Alternative Flow	If no data matches filters, system displays "No results found"
Exception Handling	If date range is invalid, system displays error and prompts for correction

UC09 – Manage User Accounts

Attribute	Description
Use Case Name	Manage User Accounts
Actor	Administrator
Precondition	Administrator is logged in
Main Flow	1. Administrator selects "User Management" 2. Administrator views the list of users 3. Administrator creates a new user or edits an existing user 4. Administrator assigns a role (e.g., rental staff, admin) 5. Administrator sets or resets user password 6. System saves the user account and role assignments
Post Condition	User accounts and roles are updated in the system
Alternative Flow	If creating duplicate username, system displays error and prompts for unique username
Exception Handling	If required fields are missing, system prevents saving and displays validation errors

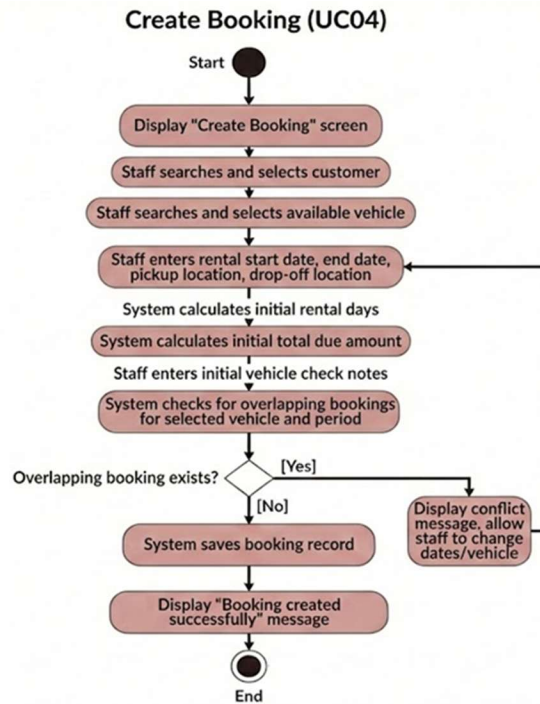
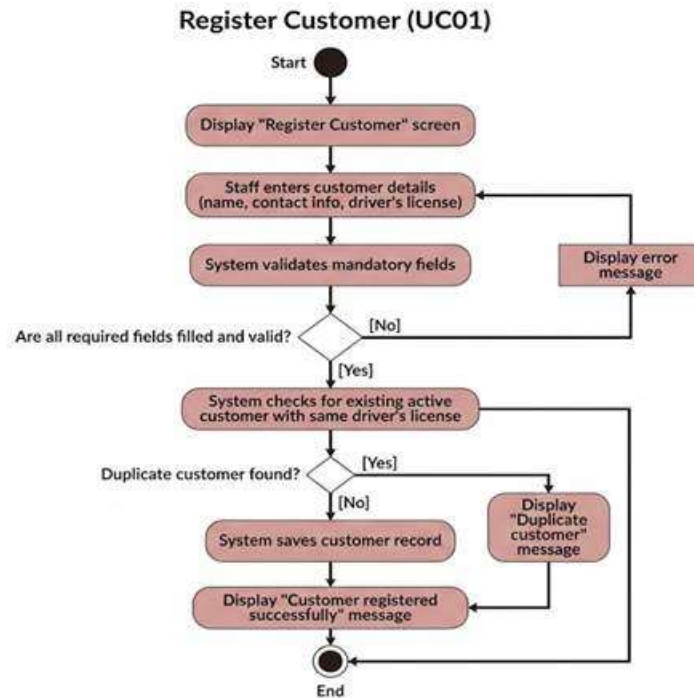
UC10 – Configure System Settings

Attribute	Description
Use Case Name	Configure System Settings
Actor	Administrator
Precondition	Administrator is logged in
Main Flow	1. Administrator selects "System Settings" 2. Administrator updates configuration values (e.g., default tax rate, late fee rules) 3. Administrator manages reference lists (vehicle categories, fuel types) 4. System validates values 5. System saves changes and confirms update
Post Condition	System settings and reference tables (e.g., categories, fuel types) are updated
Alternative Flow	If invalid configuration value entered, system displays warning and suggests valid range
Exception Handling	If changes would break existing data integrity, system prevents saving and displays error

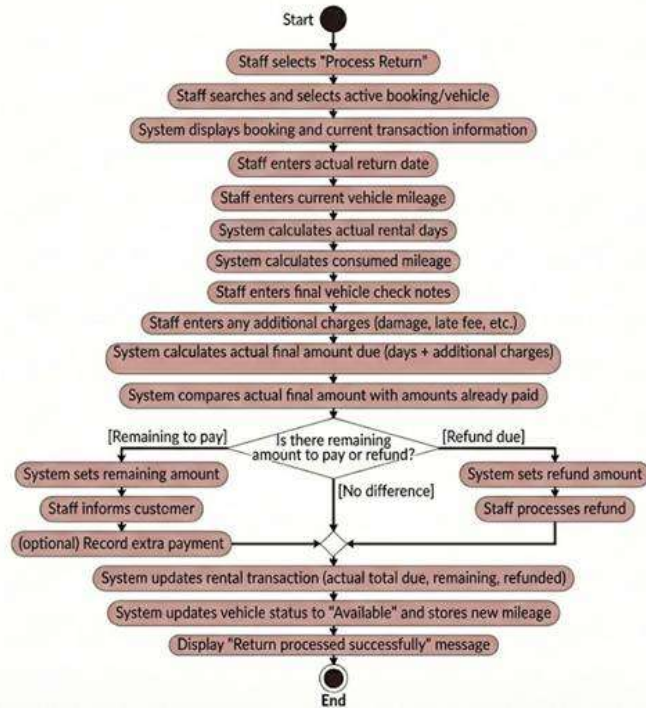
UC11 – Generate Management Reports

Attribute	Description
Use Case Name	Generate Management Reports
Actor	Administrator
Precondition	Administrator is logged in
Main Flow	1. Administrator selects "Management Reports" 2. Administrator chooses report type (e.g., monthly revenue, vehicle utilization, refund statistics) 3. Administrator selects parameters (period, branch, category) 4. System generates and displays the report 5. Administrator exports or prints the report if required
Post Condition	Management report is generated for analysis and decision making
Alternative Flow	If insufficient data available, system displays message indicating limited results
Exception Handling	If report generation fails due to data issues, system logs error and notifies administrator

9.3. Activity Diagram

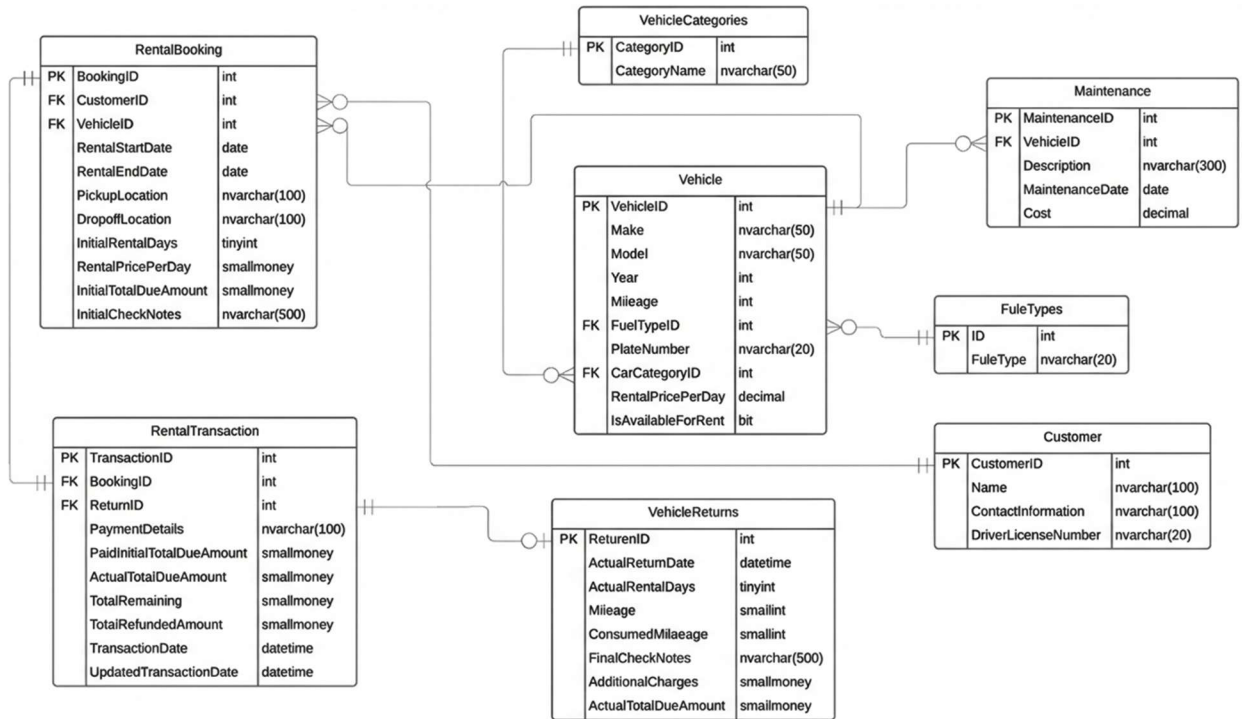


Process Vehicle Return (UC07)



8. Data Modeling

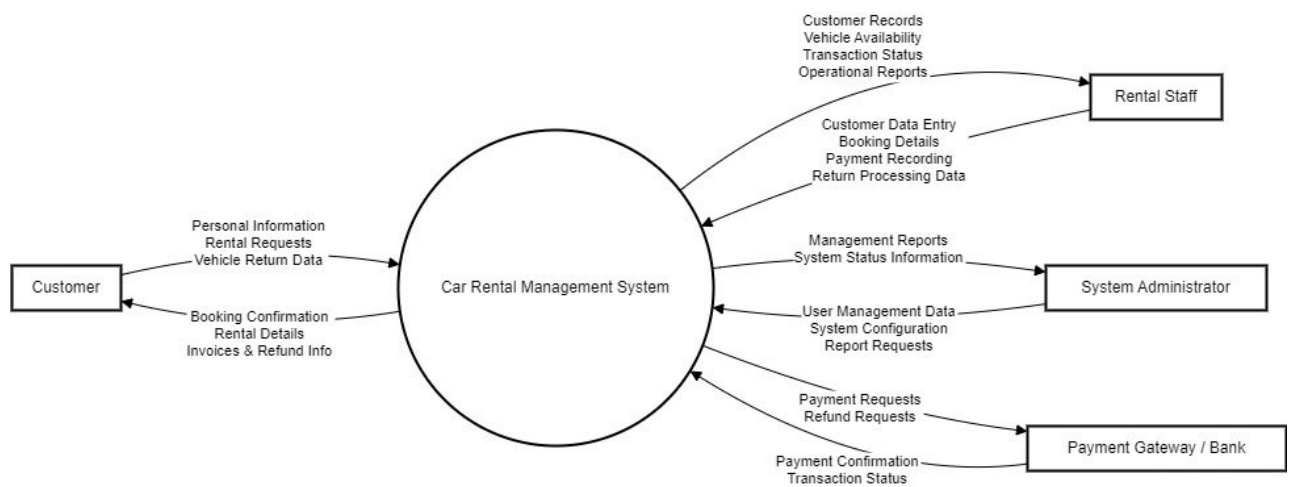
8.1. Entity Relationship Diagram (ERD)



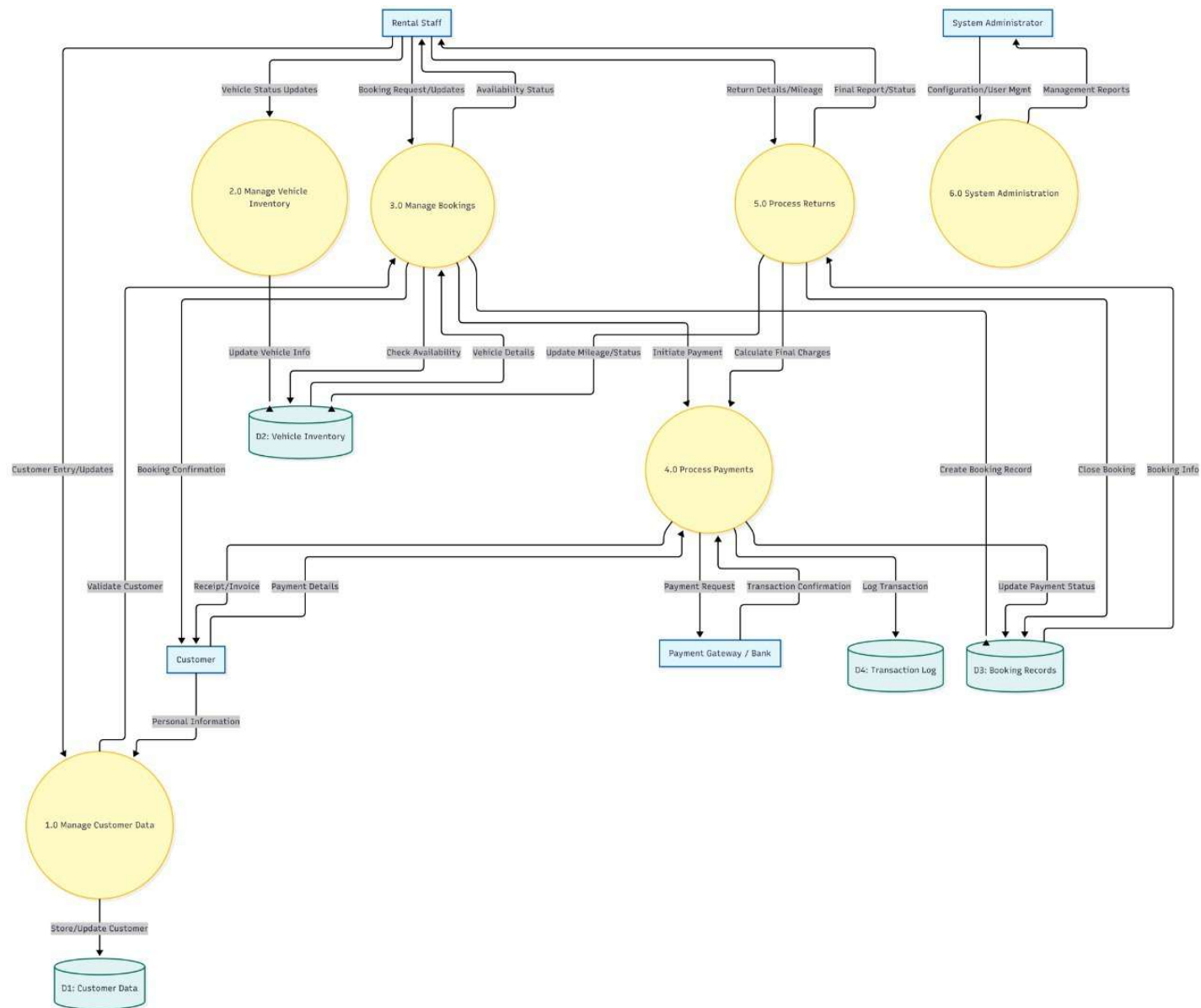
7. Process Modeling

7.1. Data Flow Diagram (DFD)

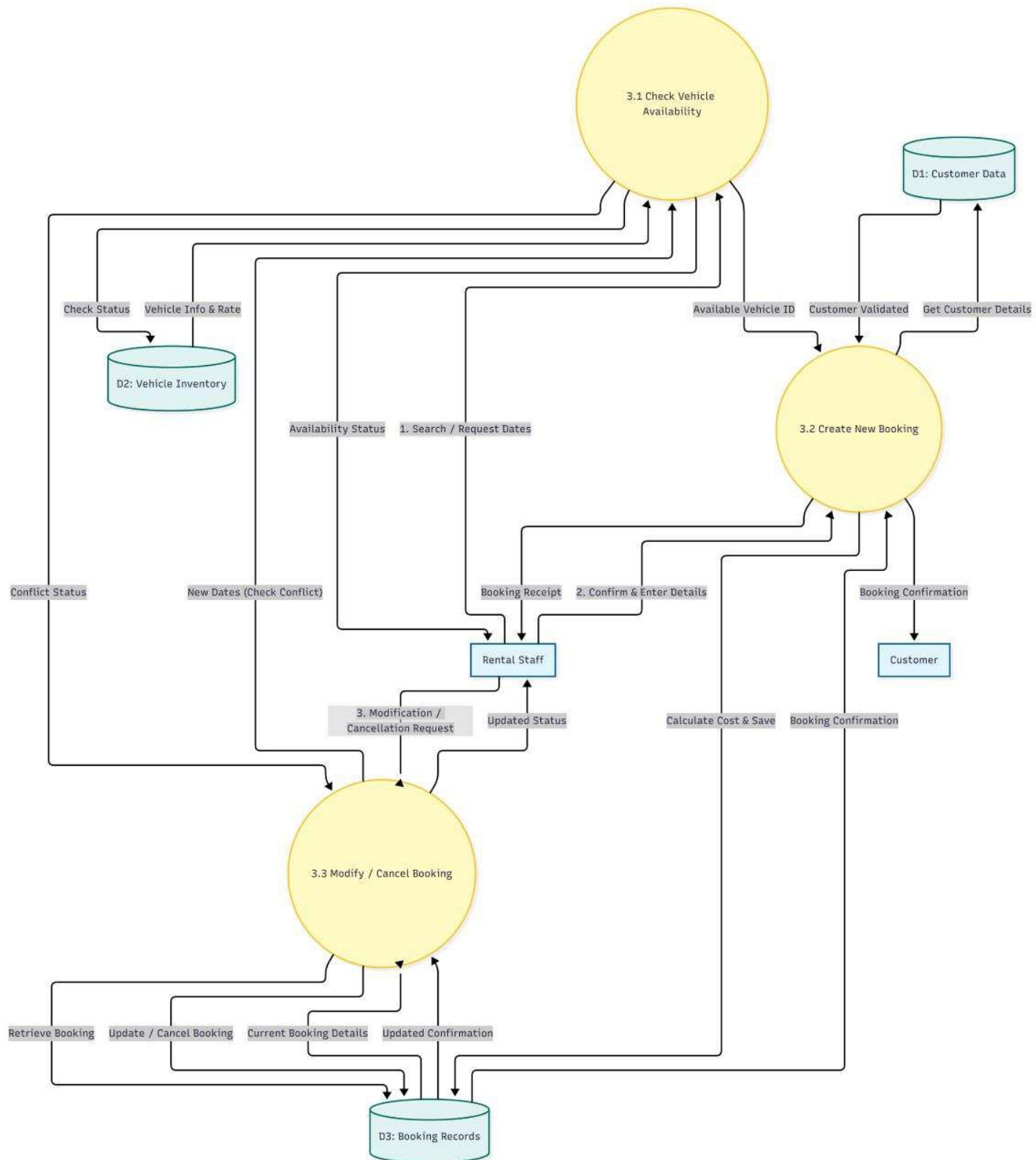
7.1.1 Context diagram



7.1.2 Level 0 diagram

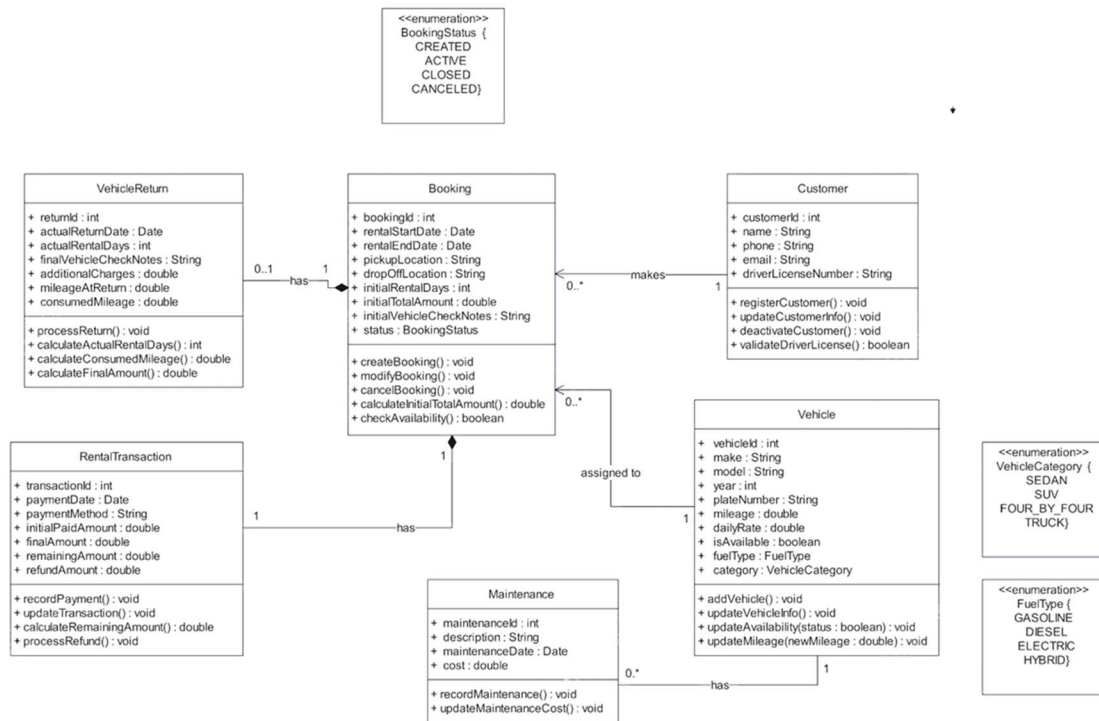


7.1.3 Level 1 diagram



10. Structural Models

10.1. Class Diagram



10.2. CRC Cards

Car Rental CRC Card

Front

Class Name: Customer	ID: 1	Type: Concrete, Domain
Description: An entity that represents a customer who rents vehicles from the car rental system.		Associated Use Cases: UC01 UC02
Responsibilities <u>Register customer</u> <u>Update customer information</u> <u>Deactivate customer</u> <u>Validate driver license number</u> 		Collaborators <u>Booking</u>

Back

Attributes: <u>customerId (int)</u> <u>name (String)</u> <u>phone (String)</u> <u>email (String)</u> <u>driverLicenseNumber (String)</u> 		
Relationships: • Generalization (a-kind-of): _____ • Aggregation (has-parts): _____ • Other Associations: _____Booking_____		

Car Rental CRC Card

Front

Class Name: Vehicle	ID: 2	Type: Concrete, Domain
Description: An entity that represents a vehicle available for rental.		Associated Use Cases: UC03
Responsibilities <u>Add vehicle</u> <u>Update vehicle information</u> <u>Update availability status</u> <u>Update mileage</u> <u>Associate maintenance records</u> <u> </u> <u> </u> <u> </u> <u> </u>		Collaborators <u>Booking</u> <u>Maintenance</u> <u> </u> <u> </u> <u> </u> <u> </u> <u> </u> <u> </u>

Back

Attributes:	
<u>vehicleId (int)</u>	<u>fuelType (FuelType)</u>
<u>make (String)</u>	<u>category (VehicleCategory)</u>
<u>model (String)</u>	<u> </u>
<u>year (int)</u>	<u> </u>
<u>plateNumber (String)</u>	<u> </u>
<u>mileage (double)</u>	<u> </u>
<u>dailyRate (double)</u>	<u> </u>
<u>isAvailable (boolean)</u>	<u> </u>
Relationships:	
• Generalization (a-kind-of): _____ • Aggregation (has-parts): _____ • Other Associations: __Booking, Maintenance_____	

Car Rental CRC Card

Front

Class Name: Booking	ID: 3	Type: Concrete, Domain
Description: An entity that manages vehicle rental reservations.		Associated Use Cases: UC04 UC05
Responsibilities Create booking Modify booking Cancel booking Calculate initial total amount Check vehicle availability 		Collaborators Customer Vehicle RentalTransaction VehicleReturn

Back

Attributes:	
bookingId (int)	status (BookingStatus)
rentalStartDate (Date)	
rentalEndDate (Date)	
pickupLocation (String)	
dropOffLocation (String)	
initialRentalDays (int)	
initialTotalAmount (double)	
initialVehicleCheckNotes (String)	
Relationships:	
- Generalization (a-kind-of): _____ - Aggregation (has-parts): _____ - Other Associations: __Customer, Vehicle, RentalTransaction, VehicleReturn_____	

Car Rental CRC Card

Front

Class Name: RentalTransaction	ID: 4	Type: Concrete, Domain
Description: An entity that handles payment processing and financial calculations for rentals.		Associated Use Cases: UC06
Responsibilities: Record payment Update transaction Calculate remaining amount Process refund 		Collaborators Booking VehicleReturn

Back

Attributes:	
transactionId (int)	
paymentDate (Date)	
paymentMethod (String)	
initialPaidAmount (double)	
finalAmount (double)	
remainingAmount (double)	
refundAmount (double)	

Relationships:
- Generalization (a-kind-of): _____ - Aggregation (has-parts): _____ - Other Associations: __Booking, VehicleReturn _____

Car Rental CRC Card

Front

Class Name: VehicleReturn	ID: 5	Type: Concrete, Domain
Description: An entity that processes the return of rented vehicles.		Associated Use Cases: UC07
Responsibilities: <u>Process vehicle return</u> <u>Calculate actual rental days</u> <u>Calculate consumed mileage</u> <u>Calculate final rental amount</u> <u> </u> <u> </u> <u> </u> <u> </u> <u> </u>		Collaborators <u>Booking</u> <u>Vehicle</u> <u>RentalTransaction</u> <u> </u> <u> </u> <u> </u> <u> </u> <u> </u>

Back

Attributes: <u>returnId (int)</u> <u>actualReturnDate (Date)</u> <u>actualRentalDays (int)</u> <u>finalVehicleCheckNotes (String)</u> <u>additionalCharges (double)</u> <u>mileageAtReturn (double)</u> <u>consumedMileage (double)</u> <u> </u> <u> </u>		<u> </u> <u> </u> <u> </u> <u> </u> <u> </u> <u> </u> <u> </u> <u> </u> <u> </u>
Relationships: - Generalization (a-kind-of): _____ - Aggregation (has-parts): _____ - Other Associations: __Booking, RentalTransaction, Vehicle_____		

Car Rental CRC Card

Front

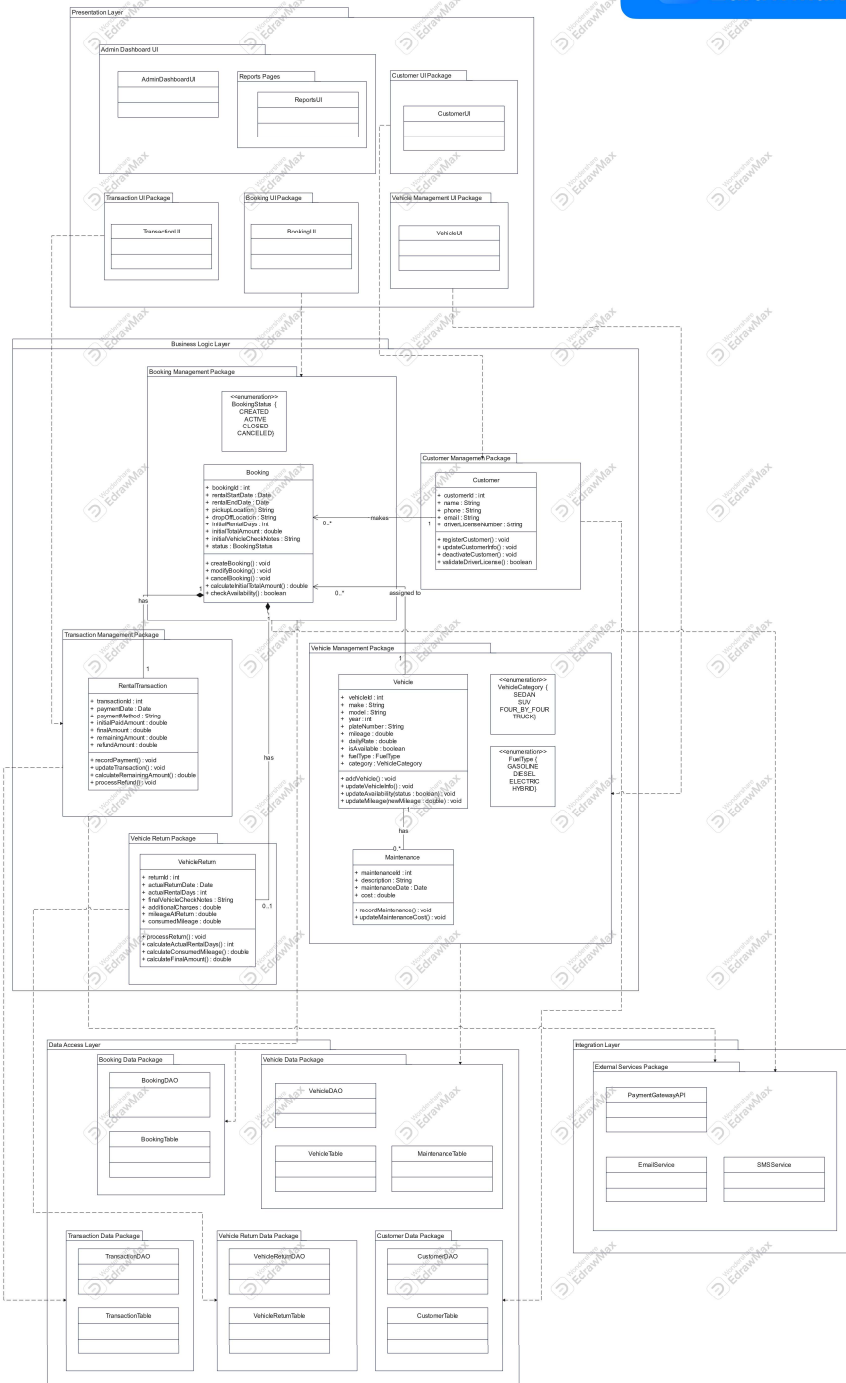
Class Name: Maintenance	ID: 6	Type: Concrete, Domain
Description: An entity that records vehicle maintenance operations.		Associated Use Cases:
Responsibilities: <u>Record maintenance</u> <u>Update maintenance cost</u> 		Collaborators <u>Vehicle</u>

Back

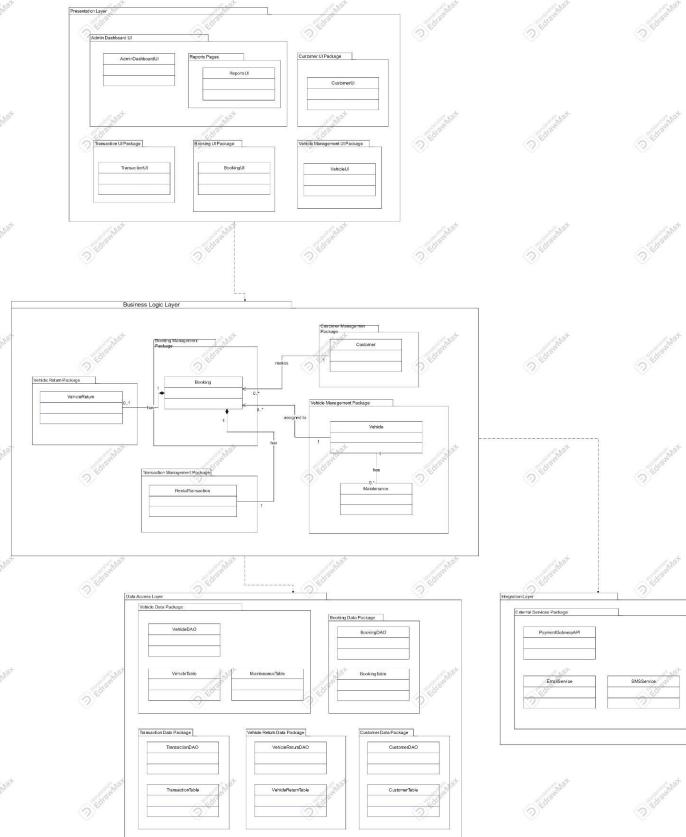
Attributes: <u>maintenanceId (int)</u> <u>description (String)</u> <u>maintenanceDate (Date)</u> <u>cost (double)</u> 		
Relationships: • Generalization (a-kind-of): _____ • Aggregation (has-parts): _____ • Other Associations: ____Vehicle____		

11. Transition to Design

11.1. Package Diagram



11.2. Package Diagram overview



12. Model Verification & Validation

12.1. Model Consistency Checklist

12.1.1 Functional Models Consistency

Business Process and Functional Modeling- Verification Rules

#	Rule Description (from Standard)	Verification Check	Status	Notes
1.1	Rule 1: When comparing an activity diagram to a use-case description, there should be at least one event recorded in the normal flow of events, sub flows, or alternative/exceptional flows of the use-case description for each activity or action that is included on an activity diagram, and each event should be associated with an activity or action.	Check if all activities match use-case events	✓	[Rule 1]
1.2	Rule 2: All objects portrayed as an object node in an activity diagram must be mentioned in an event in the normal flow of events, subflows, or alternative/exceptional flows of the use-case description.	Verify object nodes (Customer, Vehicle, Booking, etc.) appear in use cases	✓	[Rule 2]
1.3	Rule 3: Sequential order of the events in a use-case description should occur in the same sequential order of the activities contained in an activity diagram.	Compare flow order between diagrams	✓	[Rule 3]
1.4	Rule 4: When comparing a use-case description to a use-case diagram, there must be one and only one use-case description for each use case, and vice versa.	Count: Use cases = Use-case descriptions	✓	[Rule 4]
1.5	Rule 5: The use-case diagram and the activity diagram must be consistent with each other.	Check optionality and flow consistency	✓	[Rule 5]
1.6	Rule 6: All actors listed in a use-case description must be portrayed on the use-case diagram. Each actor must have an association link that connects it to the use case and must be listed with the association relationships in the use-case description.	Verify all actors connected properly	✓	[Rule 6]
1.7	Rule 7: In some organizations, we should also include the stakeholders listed in the use-case description as actors in the use-case diagram.	Check if relevant stakeholders are actors	✗	[Rule 7]
1.8	Rule 8: All other relationships listed in a use-case description (include, extend, and generalization) must be portrayed on a use-case diagram.	Verify all use-case relationships	□	[Rule 8]
1.9	Rule 9: There are many diagram-specific requirements that must be enforced. (e.g., decision node of activity diagram can be connected to activity or action nodes only with a control flow, and for every decision node there should be a matching merge node)	Check activity diagram syntax rules	✗	[Rule 9]

12.1.2 Structural Models Consistency

Structural Modeling- Verification Rules (CRC Cards ↔ Class Diagram)

#	Rule Description (from Standard)	Verification Check	Status	Notes
2.1	Rule 01: Each CRC card is associated with a class	Count: CRC cards (6) = Classes (6)	✓	Customer, Vehicle, Booking, RentalTransaction, VehicleReturn, Maintenance
2.2	Rule 02: Responsibilities on the front of the card are included as operations on the class diagram	Compare CRC responsibilities with class operations	✓	All methods must be present
2.3	Rule 03: Collaborators on the front of the card imply a relationship on the back of the card	Check CRC collaborators match relationships	✓	Dependencies must be documented
2.4	Rule 04: Attributes on the back of the card are listed as attributes on the class diagram	Compare CRC attributes with class attributes	✓	All fields must be present
2.5	Rule 05: Attributes on the back of the CRC card each have a data type (e.g., salary implies a number format)	Verify data types (int, String, Date, double, boolean, enum)	✓	No missing or incorrect types
2.6	Rule 06: Relationships on the back of the card must be properly depicted on the class diagram	Verify associations, aggregations, compositions	✓	Cardinality must be correct
2.7	Rule 07: Association classes are used only to include attributes that describe a relationship	Check if association classes used correctly	☐	If any exist, verify proper usage

12.1.3 Behavioral Models Consistency

Behavioral Modeling- Verification Rules

#	Rule Description (from Standard)	Verification Check	Status	Notes
3.1	Rule 01: Actors must be consistent between models	Compare actors across sequence, communication diagrams and use-case diagram	☐	If behavioral diagrams exist
3.2	Rule 02: Messages on sequence diagrams must match associations on communication diagrams	Cross-check messages	☐	If behavioral diagrams exist
3.3	Rule 03: Guard conditions on a sequence diagram must appear on a communication diagram	Verify guard conditions consistency	☐	If behavioral diagrams exist
3.4	Rule 04: Sequence of messages must correspond to the top down ordering of messages being sent	Check message sequence order	☐	If behavioral diagrams exist
3.5	Rule 05: State transitions must be associated with a message on a sequence diagram	Match state transitions to messages	☐	If state machines exist
3.6	Rule 06: Entries in a CRUDE matrix imply messages being sent	Verify CRUDE operations → messages	☐	If CRUDE matrix created
3.7	Rule 07: If the entry is a (C)reate, (U)pdate, or (D)eleate, then there must be an associated transition in a behavioral state machine that represents the instances of the receiving class.	Check state machines for C/U/D operations	☐	If CRUDE matrix and state machines exist

